

Financial Fraud Detection System

Machine-learning pipeline and interactive Streamlit dashboard for real-time transaction fraud scoring on extremely imbalanced data

Python

scikit-learn

SMOTE

Random Forest

Plotly

Streamlit Cloud

0.9933

ROC-AUC

71.4%

FRAUD RECALL

10,098

TRANSACTIONS

5

MODELS
COMPARED

Tanmay Patel

Data Analytics Track · July 2026

Live & Source

fraud-detection-scope.streamlit.app
github.com/tanmay866/fraud-detection

Executive Summary

Financial fraud is rare but expensive: every undetected fraudulent transaction is a direct loss, while every legitimate transaction wrongly blocked erodes customer trust. This project delivers a complete, production-shaped fraud detection system — from a raw Excel file of 10,127 transactions with only **0.67% fraud prevalence** to a publicly deployed, interactive web application.

Five machine-learning classifiers were trained on a rigorously leakage-free pipeline (train/test split *before* scaling and SMOTE resampling) and compared on an untouched held-out test set. The selected **Random Forest** model achieves **0.9933 ROC-AUC**, detecting **10 of 14 unseen fraud cases (71.4% recall)** while wrongly flagging only **5 of 2,006 legitimate transactions (66.7% precision)**. Results are served through a five-page Streamlit dashboard with live threshold tuning and real-time scoring of manually entered transactions, deployed on Streamlit Community Cloud.

Key differentiator: the pipeline's ordering — split → scale (fit on train) → SMOTE (train only) — guarantees that reported metrics reflect genuine generalization, not data leakage. This is the most common failure mode in imbalanced-data projects and was treated as the project's #1 engineering constraint.

Contents

1. Objective & Problem Statement	6. Interactive Dashboard
2. Dataset & Exploratory Analysis	7. Deployment
3. Methodology & Leakage Prevention	8. Limitations & Future Work
4. Model Training & Evaluation	9. Conclusion
5. Results & Business Interpretation	Appendix. Reproducibility Guide

1. Objective & Problem Statement

The goal was to build a two-stage system: (1) a **model pipeline** that ingests raw transaction data, handles extreme class imbalance correctly, trains and honestly compares five classifiers, and persists the best model; and (2) an **interactive dashboard** that visualizes fraud patterns, reports held-out performance, and performs real-time fraud scoring with an adjustable risk threshold — usable by non-technical stakeholders.

The central technical challenge is the class imbalance. A naive model predicting "safe" for every transaction achieves 99.3% accuracy while catching zero fraud. Every design decision — resampling, metric selection, model choice — flows from this constraint.

2. Dataset & Exploratory Analysis

Source file	Fraud_Detection.xlsx — 10,127 raw transaction records
Usable rows	10,098 after cleaning (see below)
Target	isFraud — mapped Fraud → 1, Safe → 0
Fraud cases	68 (~0.67%) — extreme class imbalance
Features	12 — transaction step, amount, 4 account-balance columns, unusual-login score, transaction type (5 categories), branch country (135), account type (2), time of day (3), day of week

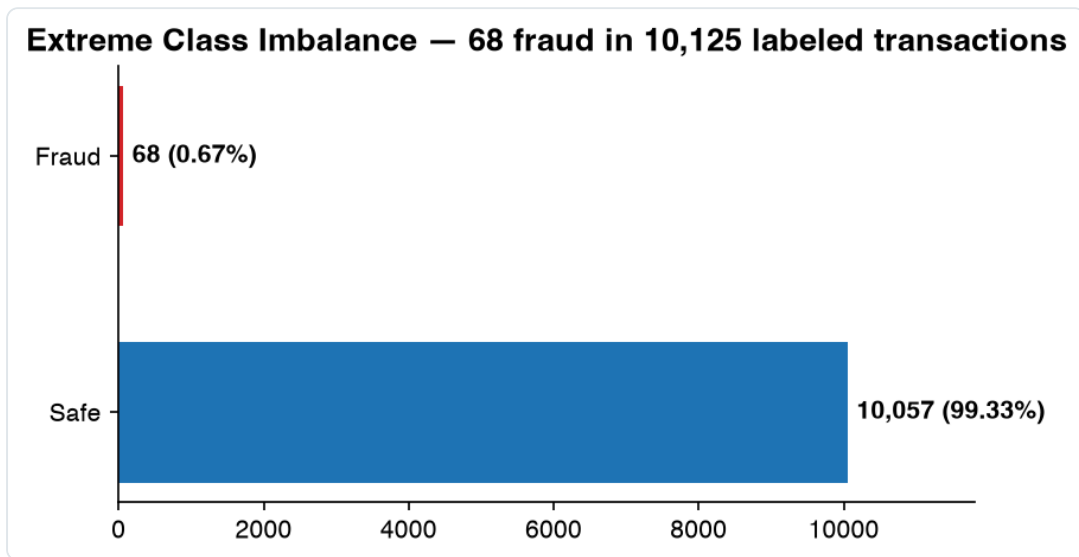


Fig. 1 — Class distribution: fraud is under 1% of all labeled transactions.

Cleaning decisions

- **2 rows** labeled "Not reviewed" dropped — no usable ground truth.
- **27 rows** with missing feature values dropped — verified that **zero** of the 68 fraud cases were lost.
- Discarded junk / leakage-prone columns: a stray duplicate of the target (`isFraud - Copy`), an index column, high-cardinality account identifiers (`nameOrig` , `nameDest`), and `isFlaggedFraud` (constant zero — carries no information).

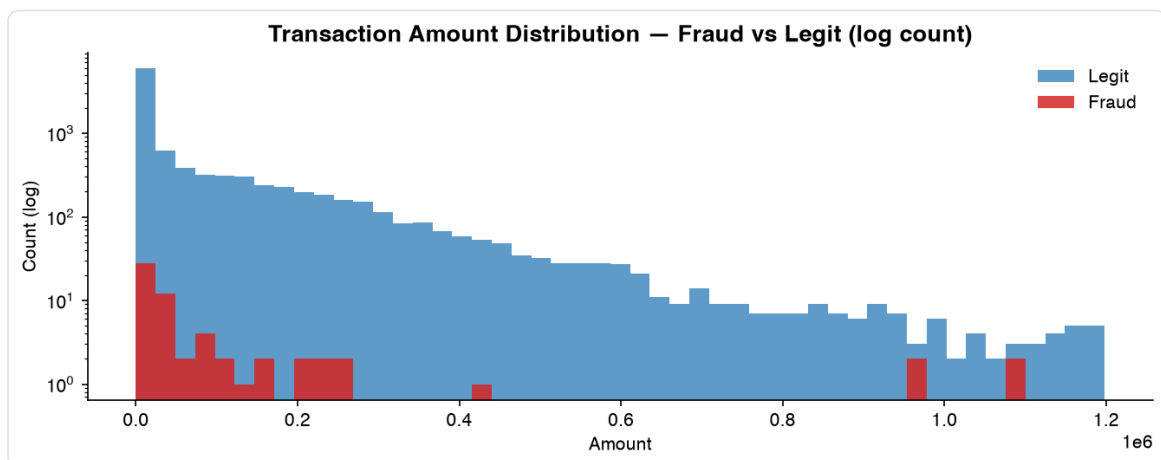


Fig. 2 — Transaction amounts, fraud vs. legit (log scale). Fraud concentrates differently across the amount range, one of the signals the model exploits.

3. Methodology & Leakage Prevention

Pipeline architecture

```
load .xlsx → filter target (Safe/Fraud only) → select 12 features → drop incomplete rows
→ label-encode 4 categoricals (encoders saved) → STRATIFIED 80/20 TRAIN/TEST SPLIT
→ RobustScaler (fit on TRAIN only, saved) → SMOTE (applied to TRAIN only)
→ train 5 models → evaluate on untouched TEST → save best model by ROC-AUC
```

Why the order matters — data leakage

The single most important design decision is that the train/test split happens **before** scaling and **before** resampling:

- **RobustScaler** (median/IQR-based, robust to the heavy-tailed balance columns) is fitted on the training split only; the test split is transformed with training statistics, never refitted.
- **SMOTE** synthesizes minority-class examples *only inside the training split*: 54 real fraud rows are expanded until the training set is balanced (8,024 / 8,024). The test set keeps its natural distribution — 2,006 safe / 14 fraud — and is never touched.

The failure mode this prevents: applying SMOTE before the split would place synthetic near-copies of test-set fraud into the training data. Models would then "recognize" test fraud they had effectively already seen, inflating every metric. All results in this report are from the untouched held-out set.

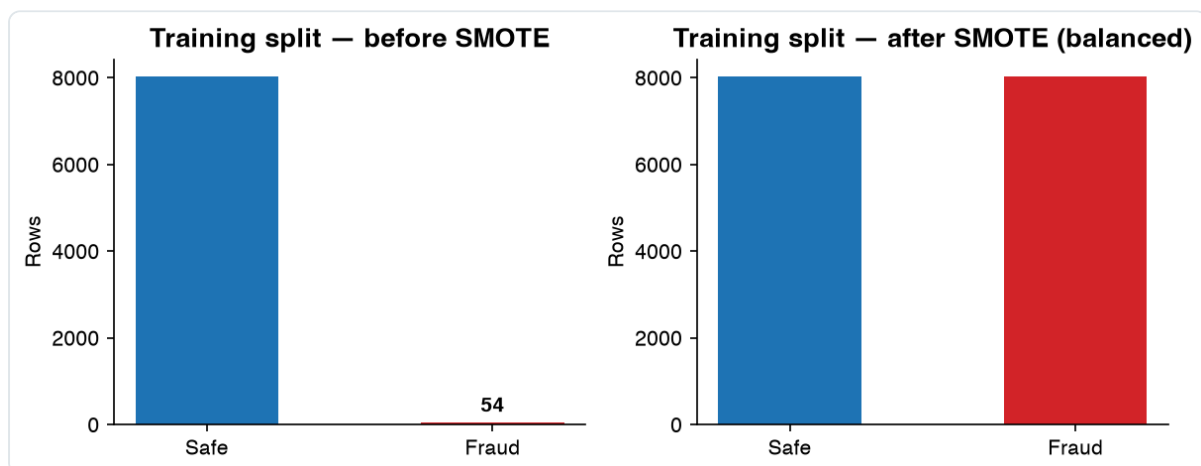


Fig. 3 — Training-split class balance before and after SMOTE. Only the training split is resampled.

Reproducibility

Every stochastic component — the stratified split, SMOTE, and all tree-based/SVC models — uses `random_state=42`. The full pipeline regenerates byte-identical artifacts from the raw dataset with three commands (Appendix). Fitted encoders and the scaler are persisted with joblib so the scoring stage and the live dashboard *transform only* and can never refit.

4. Model Training & Evaluation

Five classifiers spanning linear, instance-based, kernel, and tree-based families were trained on the SMOTE-balanced training set (16,048 rows) and evaluated on the held-out test set (2,020 rows, 14 fraud):

Model	Accuracy	Precision	Recall	F1	ROC-AUC
Logistic Regression	0.9153	0.0663	0.8571	0.1231	0.9523
K-Nearest Neighbors	0.9683	0.0833	0.3571	0.1351	0.6948
SVC (RBF kernel)	0.7876	0.0208	0.6429	0.0403	0.8147
Decision Tree	0.9901	0.3333	0.4286	0.3750	0.7113
Random Forest (100 trees) ✓	0.9955	0.6667	0.7143	0.6897	0.9933

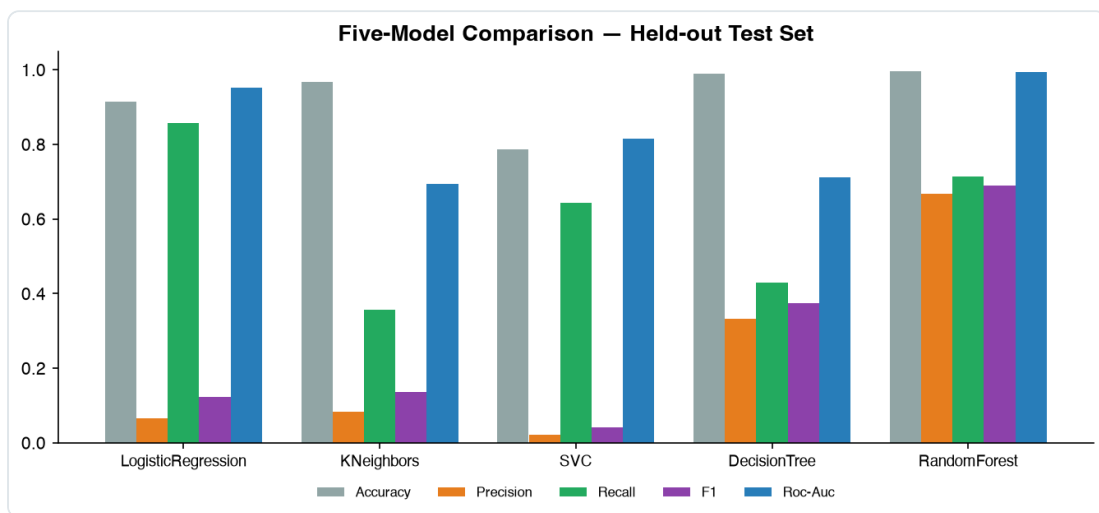


Fig. 4 — All five metrics across all five models on the held-out test set.

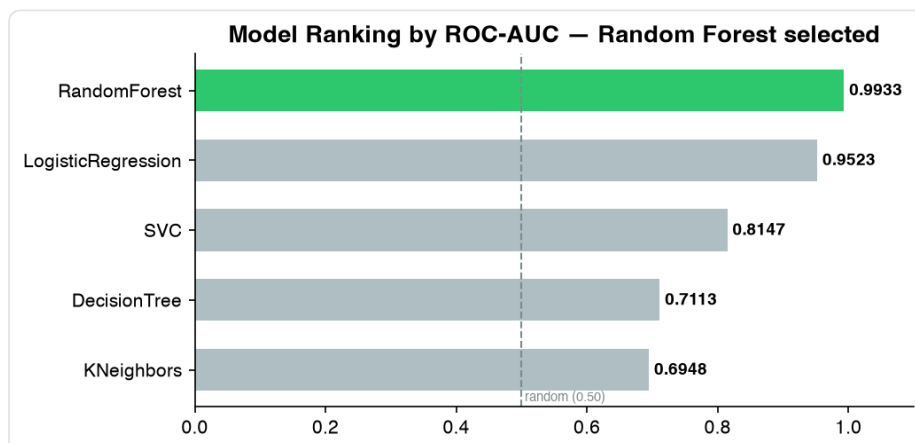


Fig. 5 — Ranking by ROC-AUC, the selection metric. Random Forest wins decisively.

Why ROC-AUC, not accuracy

With 99.3% of test transactions legitimate, accuracy rewards models for ignoring fraud entirely. ROC-AUC instead measures how well a model *rank*s fraud above legitimate transactions across all possible thresholds. Logistic Regression illustrates the danger of single metrics: its 0.95 AUC and 0.86 recall look strong, but 0.066 precision means roughly **14 false alarms for every true detection** — operationally unusable at the default threshold.

5. Results & Business Interpretation

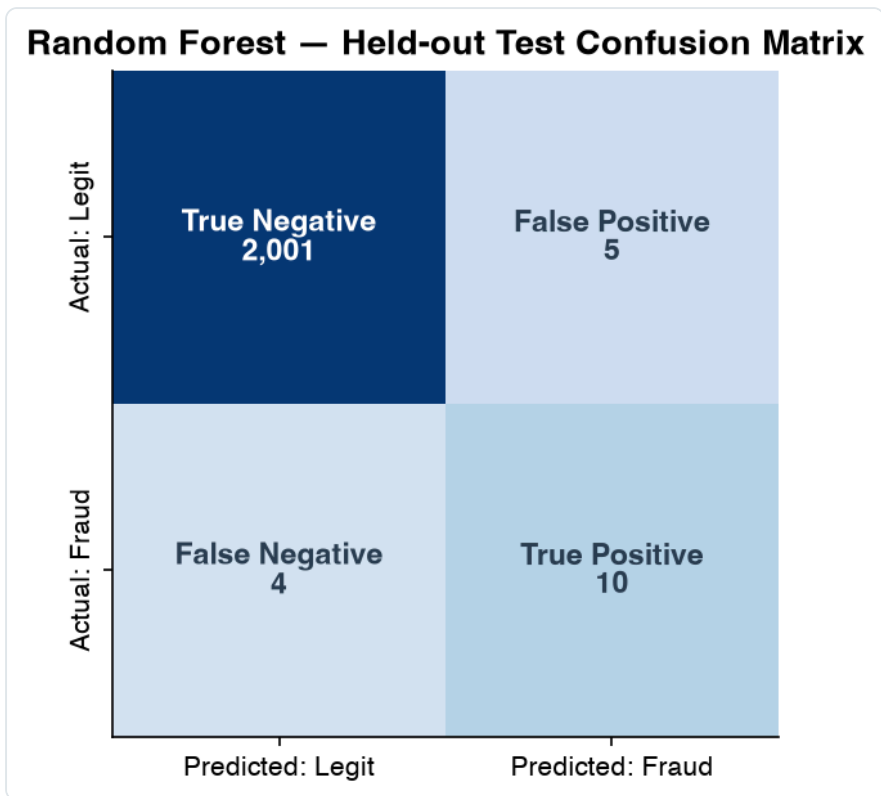


Fig. 6 — Random Forest confusion matrix on the untouched held-out test set (2,020 transactions).

Outcome	Count	Meaning for a bank
True Negatives	2,001	Legitimate transactions correctly approved
False Positives	5	Genuine customers wrongly blocked — friction, support cost
False Negatives	4	Fraud that slipped through — direct financial loss
True Positives	10	Fraud caught before completing

- **Recall 71.4%** — 10 of 14 unseen fraud cases detected.
- **Precision 66.7%** — of 15 fraud alerts raised, 10 were real; only 5 legitimate customers among 2,006 were inconvenienced.
- **Operating trade-off:** per ~2,000 transactions, the model trades 4 missed frauds against 5 blocked genuine customers — and the dashboard's live threshold slider lets an analyst move this trade-off in either direction without retraining.

Honest-metrics note: the dashboard's pattern pages score the full dataset (including training rows) for visualization richness, but every headline performance number in this report — and the dashboard's Confusion Matrix page — comes exclusively from the held-out test set, clearly labeled as such in the app.

6. Interactive Dashboard

A five-page Streamlit application serves the results. Engineering details: cached data/model loading (`st.cache_data` / `st.cache_resource`), interactive Plotly charts throughout, a consistent color language (red = fraud, blue = legit), and graceful degradation — missing artifacts produce friendly guidance, never a traceback.

Page	Contents
1 · Overview	KPI cards (transactions, fraud count/rate, model recall) + amount distribution, fraud vs legit
2 · Model Comparison	Grouped bar chart of 5 metrics × 5 models, full metrics table, best-model callout
3 · Fraud Patterns	Fraud <i>rate</i> by type / account type / time of day / day of week + day×time heatmap
4 · Confusion Matrix	Held-out test performance — annotated TN/FP/FN/TP heatmap + business explanations
5 · Transaction Explorer	Live 0–1 risk-threshold slider (re-flags in real time), filters, and a "Score a New Transaction" form running the model live

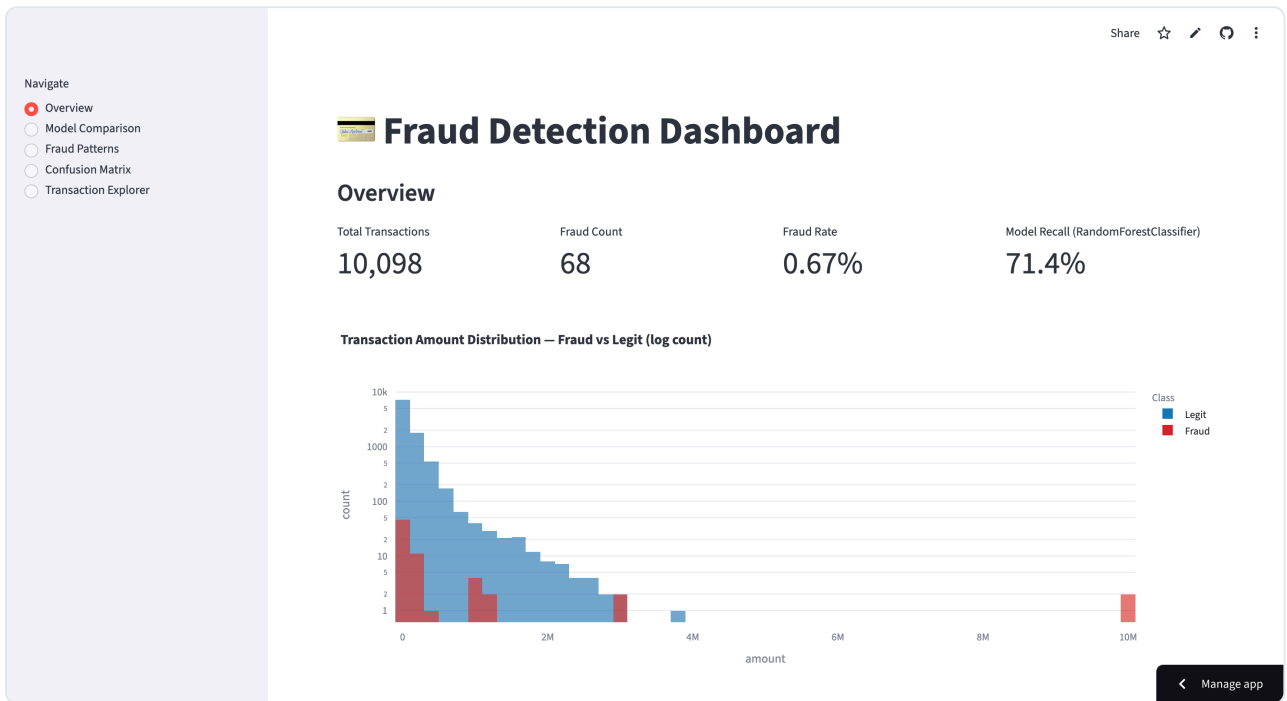


Fig. 7 — Overview page (deployed app): KPI cards and the fraud-vs-legit amount distribution.

Dashboard screenshots (continued)



Fig. 8 — Model Comparison page: all five metrics across the five models, with the best-model callout.

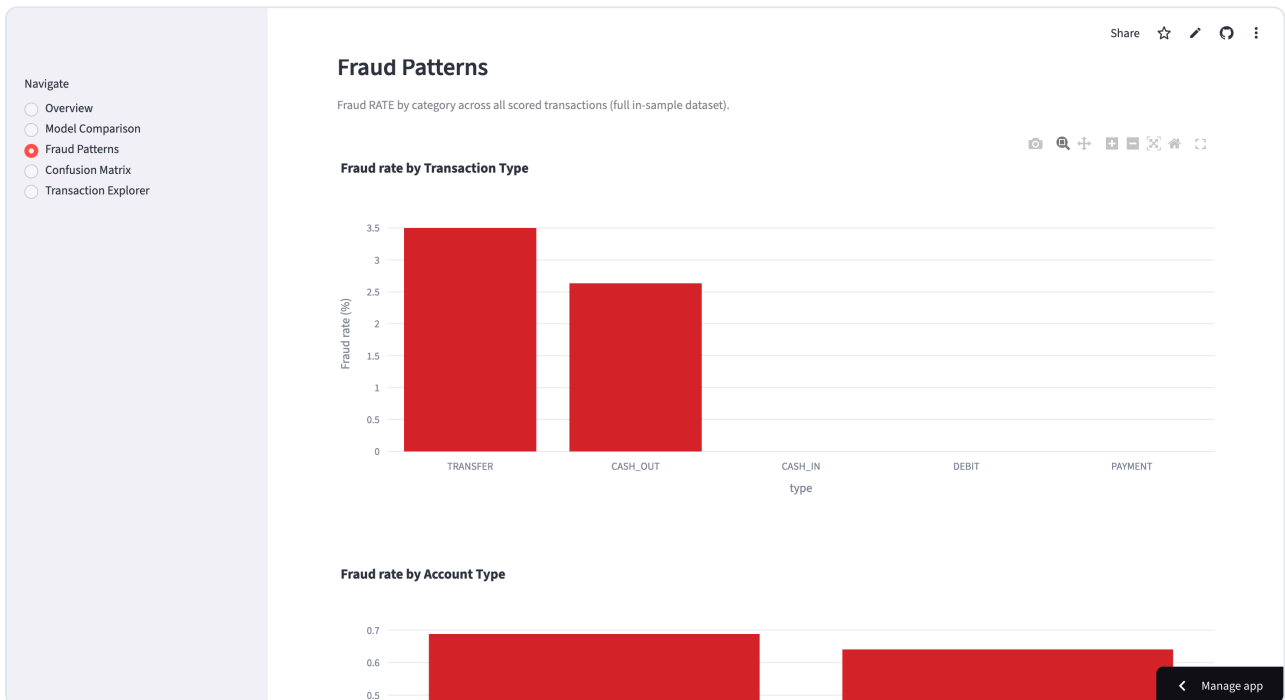


Fig. 9 — Fraud Patterns page: fraud rate by category. TRANSFER and CASH_OUT carry the highest fraud rates.

Dashboard screenshots (continued)

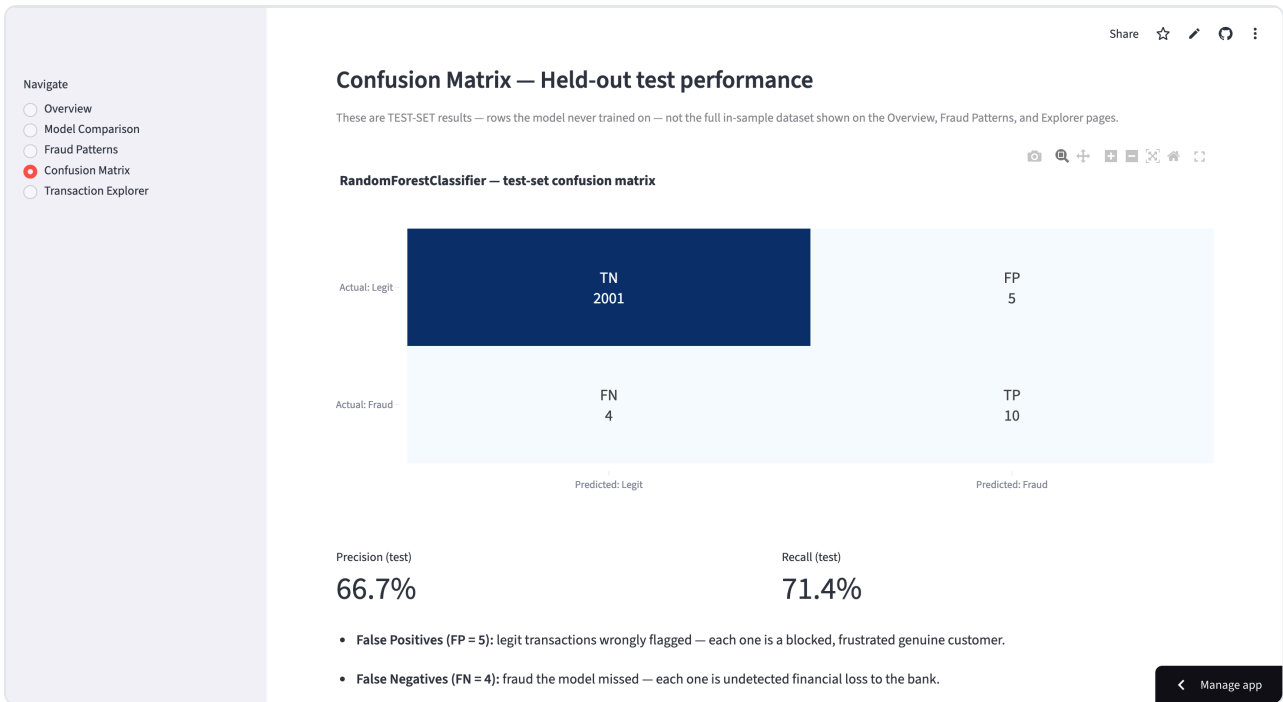


Fig. 10 — Confusion Matrix page, clearly titled "Held-out test performance", with business-language FP/FN explanations.

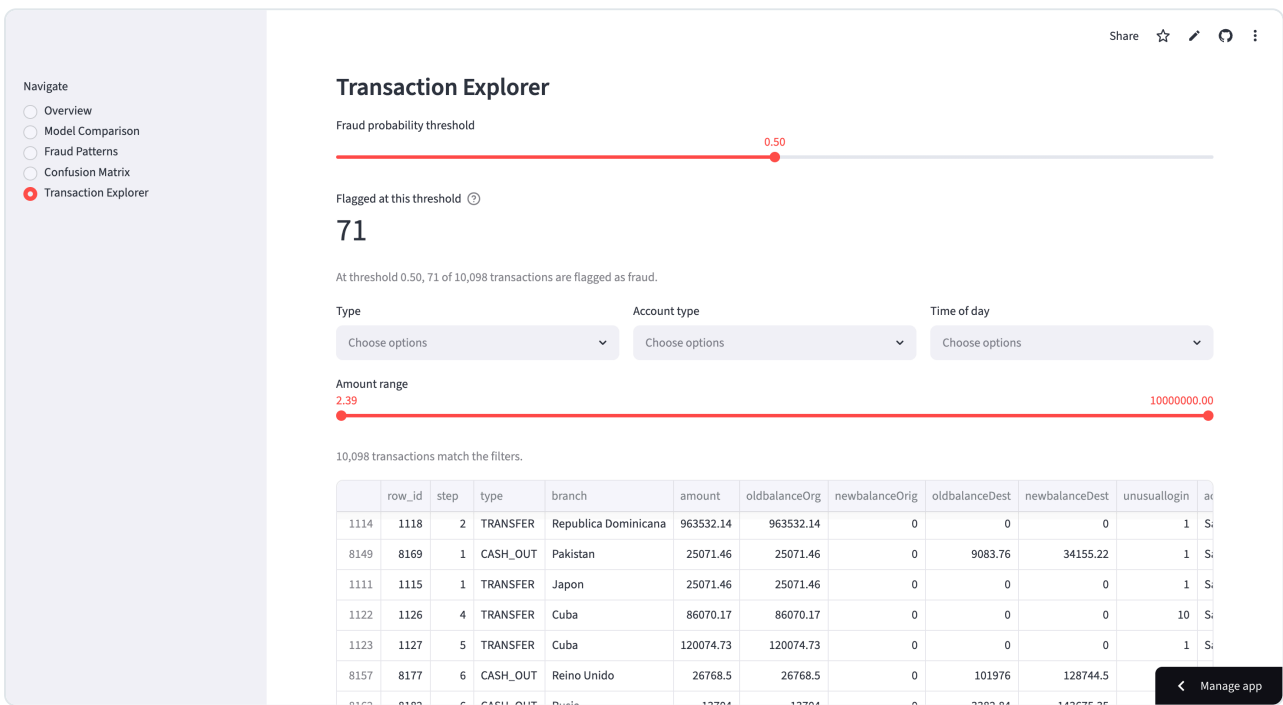


Fig. 11 — Transaction Explorer: the live risk-threshold slider re-flags transactions in real time (71 flagged at 0.50).

7. Deployment

The application is deployed on **Streamlit Community Cloud**, auto-redeploying on every push to the GitHub repository's `main` branch. Model artifacts and scored outputs are version-controlled with the code, so the cloud build is self-contained; the raw dataset itself is deliberately excluded from the repository.

Live application	https://fraud-detection-scope.streamlit.app
Source repository	https://github.com/tanmay866/fraud-detection
Stack	Python · pandas · scikit-learn · imbalanced-learn · joblib · Plotly · Streamlit

8. Limitations & Future Work

- **Small positive test sample.** With 14 fraud cases held out, each detection is worth ~7 points of recall; metrics carry natural variance. Stratified cross-validation or more labeled data would tighten the estimates.
- **Threshold optimization.** The default 0.5 threshold balances FP/FN reasonably; a cost-sensitive threshold weighing fraud loss against customer friction is the natural next step — the dashboard's slider already demonstrates the mechanics.
- **Model extensions.** Gradient-boosted trees (XGBoost/LightGBM), probability calibration, and feature-importance-guided engineering on the balance columns.
- **Production concerns.** Fraud patterns drift; scheduled retraining and input-distribution monitoring would be required in a live deployment.

9. Conclusion

This project delivered a complete, leakage-free fraud detection system: from a raw Excel file with 0.67% fraud prevalence to a deployed, interactive web application. Five classifiers were compared honestly on held-out data; the selected Random Forest achieves **0.9933 ROC-AUC with 71.4% recall and 66.7% precision** on unseen transactions. The Streamlit dashboard makes the results explorable by non-technical stakeholders, reports performance without overstatement, and demonstrates real-time scoring with an adjustable risk threshold — a defensible end-to-end template for imbalanced-classification problems in finance.

Appendix — Reproducibility Guide

```
git clone https://github.com/tanmay866/fraud-detection && cd fraud-detection
python3 -m venv .venv && source .venv/bin/activate
pip install -r requirements.txt
# place Fraud_Detection.xlsx in data/, then:
python src/preprocess.py      # shapes + class balance before/after SMOTE
python src/train.py           # 5-model comparison → metrics CSVs + best_model.pkl
python src/score.py           # full-dataset scoring → scored_transactions.csv
streamlit run app.py          # dashboard at localhost:8501
```